

Quick Guide: Setting Up Flutter and Practicing Dart Features

1. Create a Flutter Project:

- Run `flutter create dart_practice` to create a new project.
- Navigate to the project: `cd dart_practice` .
- Use the `lib/main.dart` file for Dart code or create new `.dart` files in the `lib` folder.
- To run console-based Dart code, use `dart run lib/your_file.dart` .
- To run a Flutter app, use `flutter run` with a device/emulator connected.

2. Verify Setup:

- Run `flutter doctor` to check for issues.
- Test a simple Dart file:

```
void main() {  
  print('Dart is running!');  
}
```

Run it with `dart run lib/test.dart` .

Practicing Dart Features in Flutter

Below are instructions for practicing each Dart feature category within the Flutter project. Each example includes a simple Dart program that can be run in the console (using `dart run`) or integrated into a Flutter app's `main.dart` for UI output using a `Text` widget. Files should be created in the `lib` folder of the `dart_practice` project.

Classes of Data: Primitive Types

Practice: Create a file `lib/primitive_types.dart` to experiment with `int` , `double` , `bool` , `String` , and `null` .

```

void main() {
  int score = 100;
  double price = 19.99;
  bool isActive = true;
  String name = "Dart";
  String? nullableName = null;

  print('Score: $score, Price: $price, Active: $isActive, Name: $name, Nullable: $nullableName
}

```

- **Run:** dart run lib/primitive_types.dart
- **Flutter UI:** Modify lib/main.dart to display output:

```

import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    int score = 100;
    double price = 19.99;
    bool isActive = true;
    String name = "Dart";
    String? nullableName = null;

    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text(
            'Score: $score\nPrice: $price\nActive: $isActive\nName: $name\nNullable: $null
            textAlign: TextAlign.center,
          ),
        ),
      ),
    );
  }
}

```

- **Practical Use:** Test different values for each type (e.g., negative int , empty string) and observe console/UI output. Use null to simulate missing data.

Classes of Data: Reference Types

Practice: Create `lib/reference_types.dart` to work with `List`, `Map`, `Set`, `Object?`, `dynamic`, and a user-defined class.

```
class Person {
  String name;
  Person(this.name);
}

void main() {
  List<int> numbers = [1, 2, 3];
  Map<String, int> scores = {'Alice': 90};
  Set<String> tags = {'new', 'sale'};
  Object? data = 42;
  dynamic value = "text";
  Person person = Person("John");

  print('List: $numbers, Map: $scores, Set: $tags, Object: $data, Dynamic: $value, Person: ${person.name}');
}
```

- **Run:** `dart run lib/reference_types.dart`
- **Flutter UI:** Update `lib/main.dart` to display results in a `Text` widget, similar to the primitive types example.
- **Practical Use:** Modify collections (e.g., add/remove elements) or create more complex classes to model real-world data like users or products.

Operators

Practice: Create `lib/operators.dart` to test arithmetic, comparison, logical, and other operators.

```
void main() {
  int x = 10, y = 3;
  String greeting = "Hello";
  print('Add: ${x + y}, Subtract: ${x - y}, Multiply: ${x * y}, Divide: ${x / y}, Mod: ${x % y}');
  print('Equal: ${x == y}, Not Equal: ${x != y}, Less: ${x < y}');
  print('AND: ${x > 5 && y < 5}, OR: ${x > 5 || y > 5}, NOT: ${!true}');
  String? name = null;
  print('Null Coalesce: ${name ?? "Unknown"}, Type Check: ${x is int}, Cast: ${greeting as String}');
}
```

- **Run:** `dart run lib/operators.dart`

- **Flutter UI:** Display results in a `Text` widget in `lib/main.dart`.
- **Practical Use:** Experiment with different operator combinations (e.g., `~/` for integer division) or test edge cases like `null` values with `??`.

Control Flow

Practice: Create `lib/control_flow.dart` to practice `if`, `else`, `switch`, loops, and `assert`.

```
void main() {
  int score = 75;
  if (score > 80) {
    print("High");
  } else if (score > 50) {
    print("Pass");
  } else {
    print("Fail");
  }

  switch (score ~/ 10) {
    case 8:
      print("Grade: A");
      break;
    default:
      print("Other grade");
  }

  for (var i = 0; i < 3; i++) {
    if (i == 1) continue;
    print('Loop: $i');
  }

  assert(score > 0, "Score must be positive");
}
```

- **Run:** `dart run lib/control_flow.dart`
- **Flutter UI:** Show results in a `Text` widget or use a `ListView` to display loop outputs.
- **Practical Use:** Test different conditions or loop structures, and use `assert` to catch invalid inputs during debugging.

Functions

Practice: Create `lib/functions.dart` to explore function declarations, arrow functions, and parameters.

```

int add(int a, int b) => a + b;
void greet({required String name, String? greeting = "Hi"}) {
  print('$greeting, $name');
}
var multiply = (int x, int y) => x * y;

void main() {
  print('Add: ${add(2, 3)}');
  greet(name: "Alice");
  print('Multiply: ${multiply(4, 5)}');
}

```

- **Run:** `dart run lib/functions.dart`
- **Flutter UI:** Trigger functions via `ElevatedButton` `onPress` events and display results in a `Text` widget.
- **Practical Use:** Create functions with varying parameter types (e.g., optional, named) and test them with different inputs.

Collections and Iterables

Practice: Create `lib/collections.dart` to manipulate `List`, `Map`, `Set`, and their methods.

```

void main() {
  List<int> numbers = [1, 2, 3];
  numbers.add(4);
  Map<String, int> scores = {'Alice': 90};
  scores.putIfAbsent('Bob', () => 80);
  Set<String> tags = {'new'};
  tags.add('sale');

  var doubled = numbers.map((n) => n * 2).toList();
  var sum = numbers.fold(0, (sum, n) => sum + n);

  print('List: $numbers, Map: $scores, Set: $tags, Doubled: $doubled, Sum: $sum');
}

```

- **Run:** `dart run lib/collections.dart`
- **Flutter UI:** Display collections in a `ListView` or `Text` widget.
- **Practical Use:** Experiment with methods like `where` or `remove` to filter or modify collections.

Object-Oriented Programming

Practice: Create `lib/oop.dart` to define and use classes.

```

abstract class Shape {
  void draw();
}

class Person implements Shape {
  String name;
  Person(this.name);
  Person.guest() : name = "Guest";
  factory Person.fromJson(Map json) => Person(json['name']);
  String get fullName => name;

  @override
  void draw() => print("Drawing $name");
}

void main() {
  var person = Person("John");
  var guest = Person.guest();
  var fromJson = Person.fromJson({'name': 'Alice'});
  print('Person: ${person.fullName}, Guest: ${guest.name}, From JSON: ${fromJson.name}');
}

```

- **Run:** `dart run lib/oop.dart`
- **Flutter UI:** Display class properties in a `Text` widget or use buttons to call methods like `draw`.
- **Practical Use:** Create hierarchies with `extends` or test `factory` constructors with JSON-like data.

Null Safety

Practice: Create `lib/null_safety.dart` to work with nullable types and operators.

```

void setData({required String id, String? name}) {
  print('ID: $id, Name: ${name ?? "Unknown"}');
}

void main() {
  String? nullable = null;
  late String title;
  title = "Test";

  print('Nullable: $nullable, Title: $title');
  setData(id: "123");
}

```

- **Run:** `dart run lib/null_safety.dart`
- **Flutter UI:** Use nullable fields in a form with `TextField` and display defaults with `??`.
- **Practical Use:** Test edge cases with `null` values or `!` assertions to understand null safety.

Asynchrony

Practice: Create `lib/async.dart` to handle `Future` and `Stream`.

```
Future<int> fetchData() async {
  await Future.delayed(Duration(seconds: 1));
  return 42;
}

Stream<int> countStream() async* {
  for (var i = 1; i <= 3; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;
  }
}

void main() async {
  var result = await fetchData();
  print('Fetched: $result');

  await for (var num in countStream()) {
    print('Stream: $num');
  }
}
```

- **Run:** `dart run lib/async.dart`
- **Flutter UI:** Use `FutureBuilder` or `StreamBuilder` to display async results.
- **Practical Use:** Simulate API delays with `Future.delayed` or stream real-time data.

Standard Libraries

Practice: Create `lib/libraries.dart` to use `dart:math`, `dart:io`, `dart:convert`, and `dart:async`.

```
import 'dart:math';
import 'dart:convert';
import 'dart:async';
import 'dart:io';

void main() async {
  print('Square root: ${sqrt(16)}');
  var json = jsonDecode('{"name": "John"}');
  print('JSON: $json');

  Timer(Duration(seconds: 1), () => print('Timer fired'));
  await File('output.txt').writeAsString('Hello, Dart!');
  var content = await File('output.txt').readAsString();
  print('File content: $content');
}
```

- **Run:** `dart run lib/libraries.dart`
- **Flutter UI:** Display results in a `Text` widget (note: `dart:io` is not available in Flutter web apps).
- **Practical Use:** Experiment with JSON parsing or file operations (non-web) for data processing.

Exception Handling

Practice: Create `lib/exceptions.dart` to test error handling.

```
void main() {
  try {
    var result = 1 ~/ 0;
  } on IntegerDivisionByZeroException catch (e) {
    print('Error: $e');
  } catch (e, s) {
    print('Other error: $e, Stack: $s');
  } finally {
    print('Cleanup done');
  }

  try {
    throw Exception("Custom error");
  } catch (e) {
    print(e);
  }
}
```

- **Run:** `dart run lib/exceptions.dart`
- **Flutter UI:** Show errors in a `Text` widget or dialog using `showDialog`.

- **Practical Use:** Test different exception types or custom errors to handle edge cases.

Tips for Practice

- **Console vs. UI:** Start with console output (`print`) for simplicity, then integrate into Flutter UI with `Text` or `ListView` for visualization.
- **File Organization:** Create separate `.dart` files for each category in `lib` to keep experiments organized.
- **Testing:** Use `assert` or print statements to verify behavior, and test edge cases (e.g., empty lists, null values).
- **Debugging:** Use VS Code's debugger or `debugPrint` for Flutter apps to inspect variables and flow.
- **Packages:** For advanced practice, add packages like `http` (via `pubspec.yaml`) to test real-world scenarios like API calls.